Processing Hubble Images with Blank Areas and Chip Gaps in PixInsight

A step-by-step workflow for PixInsight users with some experience. Worked example: NGC 976 from FITS-fetcher, WFC3/UVIS mosaics in F350LP, F555W and F814W (Grid A), with Pan-STARRS filling the chip gap.

Who this is for

You should already be comfortable with:

- opening images, renaming views and making previews;
- ScreenTransferFunction (STF) and HistogramTransformation (HT);
- writing simple PixelMath expressions;
- applying, showing and inverting masks.

Nothing here needs a script or a third-party tool, except where it says *optional*.

---

1. What you are dealing with

Your sensor gives you a full rectangle of data. A Hubble image doesn't:

Feature	NGC 976 example	Why it is there
Blank corners	The data is a tilted square, about 2.7′ across, inside the rectangle	The camera field is rotated on the sky, and the file is a north-up rectangle around it
Chip gap	A diagonal blank strip 30–39 px (1.2–1.6″) wide, about 40″ west of the nucleus, crossing the outer disc	WFC3/UVIS has two CCDs with a gap between them; every visit used the same orientation, so it was never filled
Noisy rim	A few pixels along every blank edge are noisier than the rest	Fewer exposures overlap at the edges of the field

FITS-fetcher saves every blank pixel as **exactly 0** (it never writes NaN) and has already trimmed the blank border to the data (header keyword `FFTRIM`). The header keyword `FFBLANK` gives the share of the image that is still blank.

Why the blank pixels cause trouble. Many PixInsight processes measure the whole image: the median and noise level for auto-stretch, the background for DBE and BackgroundNeutralization, the fit in LinearFit. A large block of zeros pulls these numbers towards 0. Multiscale processes (wavelets, noise reduction, deconvolution) treat a sharp step from data to 0 as a real structure and leave rings and halos along it.

The four rules

1. **Never let a process measure blank pixels.** Use previews inside the data, or a limit that excludes 0.
2. **Protect blank areas with a mask** while you stretch, sharpen or change colour.
3. **Re-zero the blank areas** after any process that adds an offset (DBE, normalisation, noise reduction). This is one PixelMath expression: `$T * FieldMask`.
4. **Pad before multiscale processing**: fill the blank areas with the sky level, process the image, then re-zero.

---

2. Before you start

- **Files.** FITS-fetcher names files `<Filter>_<Instrument>_<ObsID>.fits`, for example `F555W_WFC3-UVIS_hst_skycell-p1773x11y07_wfc3_uvis_f555w_all.fits`.
- **Same grid.** Images with the same Grid letter (here Grid A) share one pixel grid and have the same dimensions. **They need no registration.** Confirm that the three images have identical width and height. The `FFGRID` header keyword is the same in all three (check it with the FITSHeader process).
- **Units.** The values are in the archive's physical units: electrons per second for Hubble (`BUNIT`). Sky-subtracted sky sits near 0, so some sky pixels are **slightly negative** and bright cores can be **above 1**. PixInsight expects 0–1, so Step 3 handles this.
- **Save as XISF** as you go. XISF keeps PixInsight's astrometric solution and properties, which FITS may not.

Step 1: Open and inspect the images

1. Open the three files.
2. Rename the views (double-click the identifier tab) to **`F350LP`**, **`F555W`** and **`F814W`**. All the PixelMath expressions below use these names.
3. **Check what PixInsight did when it opened the file.** Open the Statistics process, select `F555W` and read **Minimum** and **Maximum**:
   - **Minimum below 0 or maximum above 1:** PixInsight kept the physical values. That's the expected case.
   - **Range exactly 0–1:** PixInsight rescaled the image as it opened it.
4. **Check the blank value.** Hover over a blank corner and read the readout. It should be **0**. If it shows another constant (because PixInsight rescaled the data on opening), write that value down as **`B`**. You'll test for `B` instead of 0 in Step 2.
5. **Stretch for display only, using a preview.** Don't press Auto Stretch on the whole image, because the blank corners distort the statistics. Instead:
   - draw a preview **inside the data** that includes the galaxy and some sky;
   - select the preview and press Auto Stretch (STF);
   - drag the STF *New Instance* triangle onto the main image.
6. Find the chip gap by eye. It's a thin black diagonal.

Step 2: Build the masks

You'll make three masks. Make them **now, before any process changes the values**, while blank pixels are still exactly 0.

Mask	White (1) means	Used for
`DataMask`	real data in **all three** filters, minus the noisy rim	measuring, and the edges of the gap fill
`FieldMask`	inside the camera field, **gap included**	re-zeroing and protection after the gap is filled
`GapMask`	the chip gap (plus its rim) only	filling the gap

> **Saved a data mask with FITS-fetcher?** If you ticked *Save a data mask* when downloading, each image has a `<name>_mask.fits` beside it. A folder with two or more images on one grid also has a

> `Grid_<grid>_mask.fits` with data present in every filter, which is exactly the `DataMask` of 2a. A mask file has two layers: **DATA** (white where there is data) and **GAPS** (white on the chip gap).
>
> 1. Open the Grid mask file and rename its DATA view **`DataMask`** and its GAPS view **`GapRaw`**. If PixInsight opens only the first layer, build `GapRaw` with steps 2c–2d instead.
> 2. Make `FieldMask` with PixelMath: expression `max(DataMask, GapRaw)`, *Create new image*, id **`FieldMask`**.
> 3. Do step 2b on `DataMask`, then step 2d.
>
> This skips 2a and 2c.

2a. DataMask

1. Open PixelMath.
2. **Expression (RGB/K):**

   ```
   iif(F350LP == 0 || F555W == 0 || F814W == 0, 0, 1)
   ```

   If Step 1 gave you a blank value `B` other than 0, use `abs(F555W - B) < 1e-7` and so on in place of each `== 0` test.
3. **Destination:** *Create new image*, Image id **`DataMask`**, Color space **Gray**.
4. Apply it to any one of the three images (drag the triangle onto `F555W`).

2b. Pull the mask in from the noisy rim

1. Open MorphologicalTransformation.
2. Set **Operator: Erosion (minimum)**, **Structuring element: 7 × 7 circular**, **Iterations: 1**, **Amount: 1**.
3. Apply it to `DataMask`. The white area now stops 3 px short of every blank edge, including both sides of the chip gap.

2c. FieldMask (close the gap)

1. Duplicate `DataMask` (drag its identifier tab onto the workspace) and rename the copy **`FieldMask`**.
2. In MorphologicalTransformation, set **Operator: Dilation (maximum)**, **25 × 25 circular**, **Iterations: 2**. Apply it to `FieldMask`.
3. Set **Operator: Erosion (minimum)** with the same size and **Iterations: 2**. Apply it to `FieldMask` again.

Dilating and then eroding by the same amount closes any blank strip narrower than about 48 px, which covers the 30–39 px gap. The outer edge returns to where it was, apart from slightly rounded inside corners.

> If your PixInsight version limits the structuring element to a smaller size, raise the number of iterations so that size × iterations is still about 50 px, and use the same setting for both operations.

2d. GapMask

1. In PixelMath, use the expression `max(FieldMask - DataMask, 0)`.
2. Set Destination to *Create new image* with Image id **`GapMask`**, Gray, and apply it.
3. Check the result. You should see **only the diagonal strip** of the gap, plus perhaps a few pixels in inside corners. If you see the whole outer border, step 2c didn't run both operations.

> **No gap fill planned?** Skip `GapMask`. Use `DataMask` wherever this guide says `FieldMask`; the gap then stays black.

Step 3: Bring the values into PixInsight's 0–1 range

Skip this step if Statistics showed the images already in 0–1 (Step 1.3).

For each image:

1. In Statistics, read **Minimum** and **Maximum** for the whole image and write them down as `lo` and `hi`.
2. In PixelMath, select **Replace target image**, and under *Destination* turn **Truncate on** and **Rescale result off**. Apply this expression to the image:

   ```
   iif(FieldMask == 0, 0, ($T - lo) / (hi - lo))
   ```

   Type the numbers you wrote down in place of `lo` and `hi`.

This maps the most negative sky pixel to 0 and the brightest core to 1, and keeps every blank pixel at exactly 0. Because the scaling is linear, the data is still linear.

If PixInsight had already rescaled the images when it opened them, run only `$T * FieldMask` to set the blank value `B` back to 0.

Step 4 (optional): Fill the chip gap with Pan-STARRS

A gap only a few pixels wide in empty sky can be handled with CloneStamp. Here it crosses the galaxy's disc, so it's better to fill it with real data. In FITS-fetcher's NGC 976 search, the **Pan-STARRS GPC1 set** (g, r, i, z, y on one grid, a 2.4′ box at 0.25″/px) covers the whole galaxy and has no blank pixels along the gap.

The fill will be **softer** than the Hubble data around it (seeing about 1.1″ against 0.1″). Over a 1.4″ strip in the outer disc this is hard to see; over a compact knot you may see it.

4a. Download and prepare

1. Download the PS1 **g, r, i and z** images (y isn't needed).
2. Open them, rename them **`g`**, **`r`**, **`i`** and **`z`**, and check their range with Statistics. FITS-fetcher's PS1 images are already linearised. If they are outside 0–1, normalise them as in Step 3, but **without** the `FieldMask` term.
3. Upsample them so their scale is close to Hubble's. In Resample, set **Mode: Relative, 625 %**, **Interpolation: Lanczos-3** (or Auto) and apply it to all four. This makes registration far easier for StarAlignment.

4b. Register PS1 onto the Hubble grid

1. Open StarAlignment and set **Reference image** to the view `F555W`. Add `g`, `r`, `i` and `z` as targets, or register the open views one at a time.
2. Set **Registration model: Projective** and **Interpolation: Lanczos-3** (or Auto). Leave drizzle data off.
3. Run it. Each registered image has exactly the Hubble image's size.
4. Check the alignment by blinking a registered PS1 image against `F555W`. Stars and knots should not move.

If StarAlignment can't match stars (3′ fields have few of them), there are two other routes:

- **Raise the star detection sensitivity** in StarAlignment's *Star Detection* section and try again.
- **Align by coordinates instead:**
- Save both images as XISF.
- Make sure each has an astrometric solution. FITS-fetcher writes the archive's WCS, but if PixInsight doesn't show a solution, run ImageSolver.

- Use the AlignByCoordinates script with `F555W` as the reference.

4c. Make PS1 images that match each Hubble filter

Use PixelMath with *Create new image*, after renaming the registered results back to `g`, `r`, `i` and `z`:

New image id	Expression	Stands in for
`PS1_V`	`0.4*g + 0.6*r`	F555W (V band, between g and r)
`PS1_I`	`0.6*i + 0.4*z`	F814W (700–950 nm)
`PS1_W`	`(g + r + i + z) / 4`	F350LP (white light, about 350–950 nm)

The weights are approximate. The next step fixes the overall brightness, and any colour difference left over a 1.4″ strip is small.

4d. Match brightness with LinearFit

For each pair (`F555W` with `PS1_V`, `F814W` with `PS1_I`, `F350LP` with `PS1_W`):

1. In LinearFit, set **Reference image** to the Hubble view (for example `F555W`).
2. Set **Reject low** to a value just above 0, such as `0.00001`, so blank pixels and the area outside the PS1 box are left out of the fit. Leave **Reject high** at its default, which keeps star cores and the nucleus out of it.
3. Apply it to the PS1 image (for example `PS1_V`).
4. Check the result: with STF linked and the same stretch on both, the PS1 image's disc should look about as bright as the Hubble one beside the gap.

4e. Blend the PS1 data into the gap

1. Make a soft-edged copy of the gap mask. Duplicate `GapMask` and rename the copy **`GapSoft`**. Apply Convolution to it with **Gaussian, StdDev 3 px**.
2. Open PixelMath. In **Symbols**, enter `w`. In **Expression**, enter:

   ```
   w = min(1, max(GapMask, GapSoft));
   ($T*(1 - w) + PS1_V*w) * FieldMask
   ```

3. Select **Replace target image** and apply it to `F555W`.
4. Repeat for `F814W` with `PS1_I` and for `F350LP` with `PS1_W`, changing only the PS1 name.

   Inside the gap, `w` is 1, so the pixel comes entirely from PS1. Over the next few pixels it fades to 0, so the seam is soft. Outside the field, `FieldMask` keeps everything at 0.
5. Inspect the gap at 1:1 and 2:1 zoom. If the seam shows as a brighter or darker line, LinearFit's fit was off. Redo 4d with a preview drawn along the gap, using that preview as the image LinearFit measures.

From here on, **`FieldMask` is your data mask**, because the gap is now filled.

Step 5: Background (usually small for Hubble)

HAP mosaics are sky-subtracted, so large gradients are rare. If you do see one:

1. Open DynamicBackgroundExtraction (DBE) and place samples **by hand on empty sky inside the field**. Delete any sample that lands on a blank area or on the galaxy.
2. Set **Correction: Subtraction** and run it.

3. **Re-zero:** in PixelMath, apply `$T * FieldMask` to the corrected image. DBE adds an offset that would otherwise lift the blank areas off 0.

Step 6: Combine colour (still linear)

With three filters, the usual Hubble mapping is:

Channel	From
Red	`F814W`
Green	`(F555W + F814W) / 2`: a synthetic green made with PixelMath, new image `G_syn`
Blue	`F555W`
Luminance (later)	`F350LP`, the deepest image (about 16,700 s)

1. In ChannelCombination, choose **RGB**, set R = `F814W`, G = `G_syn` and B = `F555W`, and apply it. Rename the result **`RGB`**.
2. Re-zero with `$T * FieldMask`. PixelMath applies a single gray image to all three channels.
3. **BackgroundNeutralization:** set the reference image to a **preview on empty sky inside the field** and apply it to `RGB`.
4. **ColorCalibration:**
   - **White reference:** a preview around **the whole galaxy**. An integrated spiral galaxy is a good white reference. Turn *Structure detection* off.
   - **Background reference:** the same sky preview as in BackgroundNeutralization.
5. Re-zero with `$T * FieldMask`.

> SpectrophotometricColorCalibration (SPCC) needs plenty of catalogue stars. A 3′ Hubble field rarely has enough, which is why the galaxy is used as the white reference above.

Step 7: Linear noise reduction and sharpening, with padding

Do this on `RGB` and on `F350LP`. Skip it if you're happy with the noise.

1. **Measure the sky.** Draw a preview on empty sky inside the field. In Statistics, write down its **median** as `S` (for `RGB`, one value per channel, or their average).
2. **Pad.** In PixelMath, apply `iif(FieldMask == 0, S, $T)`, with your number in place of `S`. The blank areas now look like sky, so there's no edge left to ring.
3. **Process.** Use MultiscaleLinearTransform or TGVDenoise. *Optional:* NoiseXTerminator or BlurXTerminator, if you have them.
4. **Re-zero.** Apply `$T * FieldMask`.

Step 8: Stretch

1. Draw a preview inside the field that includes the galaxy and sky.
2. With the preview selected, press Auto Stretch in STF.
3. Drag the STF *New Instance* triangle onto the **bottom bar of HistogramTransformation** to copy the settings.

4. Reset STF on the main image, then apply HistogramTransformation to the **main image**.
5. Do the same for `F350LP` (the luminance).

HistogramTransformation maps 0 to 0, so the blank areas stay black. GeneralizedHyperbolicStretch works just as well if you prefer it; measure on the same preview.

> The stretch is the step that goes most wrong with blank pixels. If the image comes out grey or washed out, you measured on the whole image instead of the preview.

Step 9: Add luminance, then the non-linear steps

1. **LRGBCombination:** select `F350LP` as **L**, untick R, G and B, and apply it to `RGB`.
2. **Apply `FieldMask` as a mask** to the result, with *Show mask* off. Everything you do now affects only the field:
   - CurvesTransformation for contrast and saturation;
   - HDRMultiscaleTransform on the bright core. Add a range mask if you want, but combine it with `FieldMask`, for example `min(RangeMask, FieldMask)`;
   - LocalHistogramEqualization for the spiral arms.
3. Remove the mask and do a final re-zero with `$T * FieldMask`.

Step 10: Choose how to finish the edges

Option	How	Result
Keep the true shape	Nothing more to do	A tilted, slightly jagged field on black, in the tradition of the early Hubble releases
Crop to a rectangle	DynamicCrop: show `FieldMask` beside the image and draw the largest rectangle that stays inside the white area	A clean rectangle. For NGC 976 this costs part of the outer disc
Fill the corners from Pan-STARRS	Step 4, but with `1 - FieldMask` instead of `GapMask`, over the area the PS1 box covers	A full rectangle with soft ground-based corners, the technique of Hubble-plus-ground composites

Save the final image as **XISF**, then export TIFF or JPEG as needed.

Quick reference

Symptom	Cause	Fix
Auto Stretch makes the image grey or white	Blank pixels in the statistics	Stretch from a preview inside the data (Steps 1.5 and 8)
Bright or dark halo along the field edge or gap after wavelets or noise reduction	Step from data to 0	Pad with the sky level first (Step 7)
Blank areas turn grey after DBE or BackgroundNeutralization	The process added an offset	`$T * FieldMask`
LinearFit gives an absurd slope	Zeros included in the fit	Set **Reject low** just above 0 (Step 4d)
StarAlignment finds no matches with PS1	Big scale difference, few stars	Resample PS1 by 625 % first, or use AlignByCoordinates (Step 4b)
The `== 0` tests make an all-white mask	PixelMath or PixInsight rescaled the data, so blank isn't 0	Use the blank value `B` from the readout (Step 1.4)
Seam visible along the filled gap	Brightness mismatch	Redo LinearFit on a preview along the gap (Step 4e.5)

A note on star cores

If FITS-fetcher's *Repair star cores* option was on (`FFREPAIR = T`), small saturated holes in star cores were already filled before saving. Chip gaps and the blank border are never "repaired"; the masks above are what deal with them.

Written for FITS-fetcher v0.1 output (spec v1.23), 2026-10-21. Process and option names follow PixInsight 1.8.9 and later. Where your version labels an option differently, the intent described here still applies.